Software Requirements Specification (SRS) for a Risk, Back-testing, Portfolio Optimization & TCA System (RPAS)

Software Requirements Specification (SRS) is the comprehensive blueprint for building a new software system. It is the single source of truth for all stakeholders—from developers to end-users—defining exactly what the system should do, how it should behave, and what its limitations are. The document for the Risk, Back-testing, Portfolio Optimization & TCA System (RPAS) is a prime example, meticulously detailing the project's scope, functionality, and constraints.

1. Introduction: Setting the Stage

This section provides the high-level context, establishing the project's purpose and scope. It's the "elevator pitch" of the document.

1.1 Purpose:

The core objective of the RPAS is to empower investment teams with the tools to make data-driven decisions. Instead of relying on gut feelings, they can use the system to:

Quantify Risk: Understand potential losses in worst-case scenarios and ensure portfolios remain within acceptable risk limits. This helps prevent catastrophic financial events.

Validate Strategies: Use historical data to test new trading ideas, allowing quants and researchers to refine their approaches without risking real capital.

Enhance Performance: Optimize portfolio allocations to maximize returns while managing risk, moving from a current, inefficient state to a more optimal one.

Reduce Costs: Analyze the hidden fees of trading—like slippage and commissions—to identify and eliminate inefficiencies that erode profits over time.

This section clearly translates complex financial concepts into actionable goals, making the purpose of the system clear to all stakeholders, whether they are a seasoned portfolio manager or a new developer.

1.2 Scope: What's In, What's Out

This is a critical section for managing expectations and preventing scope creep—the uncontrolled expansion of project requirements.

In Scope (What the System Will Do): The list is a detailed breakdown of the system's core functionalities. It specifies concrete deliverables, such as calculating Value at Risk (VaR) and

Expected Shortfall (ES) at specific confidence levels (95\% and 99\%). It also defines the types of stress tests, like a "+200 basis points interest rate shock," and the types of strategies to be back-tested (e.g., momentum and mean reversion). The inclusion of reproducibility is a key feature, ensuring that any analysis can be rerun with the exact same inputs to verify results, which is crucial for auditing.

Out of Scope (What the System Will NOT Do): Equally important, this list explicitly states what the system will not do. For example, it won't handle live trading or market predictions, which are separate, complex systems in their own right. This keeps the project focused on its core analytical purpose.

1.3 Definitions and Acronyms: The Language of the Project

This section acts as a glossary, defining all specialized financial and technical terms. This ensures that everyone reading the SRS, regardless of their background, has a shared understanding of the concepts. For example, it explains:

* VaR (Value at Risk): The maximum probable loss over a specific time period. The 95\% VaR means there's only a 5\% chance of losing more than that amount.

* Sharpe Ratio: A measure of a portfolio's return relative to its risk. A higher Sharpe ratio indicates a better risk-adjusted performance.

* Slippage: The difference between the expected price of a trade and the price at which the trade is actually executed.

2. Overall Description: A Broader View

This section places the system within the context of the organization, describing its function and its users.

2.1 Product Perspective & 2.2 Product Functions

The RPAS is designed as a standalone analytical engine. It is not integrated directly into trading platforms but rather consumes data from them. Its main functions are distinct but interconnected:

Risk Analytics: A daily process that generates a snapshot of portfolio risk.

Back-testing: An on-demand service for research and validation.

Portfolio Optimization: A decision-support tool to guide portfolio rebalancing.

TCA: A recurring analysis to measure and improve trading efficiency.

Reporting: The primary output that consolidates all analytical results.

2.3 User Characteristics & 2.4 Principal Actors

This section identifies the system's users and their specific needs. This understanding is critical for designing an intuitive user interface and generating relevant reports.

Risk Managers: Need clear, concise reports on risk metrics like VaR and ES, along with real-time alerts for breaches.

Portfolio Managers: Are interested in trade recommendations from the optimization module and daily performance reports.

Quants/Researchers: Require a flexible interface to configure and run complex backtests.

Execution Teams: Focus on the weekly TCA reports to identify costly brokers or algorithms.

Auditors: Require the system's auditability and reproducibility features to verify historical results.

The document correctly assumes that these users have a solid grasp of financial concepts but may not have programming expertise, which influences the system's design toward user-friendly dashboards rather than complex command-line interfaces.

2.5 General Constraints & 2.6 Assumptions and Dependencies

These sections list the non-functional requirements and external factors that must be considered.

* Constraints: These are the limitations, such as a strict timeline for daily jobs ("must finish within 30 minutes of data arrival") and the need to handle a large volume of data ("10,000+ positions").

* Assumptions and Dependencies: These are external conditions that the project relies on, such as the consistent availability of reliable market data and the necessary computing resources.

3. Specific Requirements: The Core Functionality

This is the most detailed part of the SRS, providing a granular list of what the system must do.

3.1 Functional Requirements: A Detailed Breakdown

This section is organized by module, providing precise descriptions of each function.

A. Data Ingestion: Beyond simply loading data, the system must validate it (e.g., checking for missing values) and save snapshots of all inputs. This is crucial for auditability and reproducibility.

B. Back-testing: The system must not only test strategies but also model realistic trading costs like slippage and market impact. This ensures that the backtest results are not just theoretical but reflect real-world performance.

C. Portfolio Optimization: The system must solve a complex mathematical problem to find the optimal portfolio under various constraints, such as turnover limits (how much the portfolio can be changed) and sector limits. It must then provide concrete trade recommendations to move from the current portfolio to the target.

D. Transaction Cost Analysis (TCA): This module goes beyond simple cost calculation. It will attribute P&L (Profit & Loss) to different factors like alpha (manager skill), beta (market exposure), cost, and timing. This advanced analysis helps pinpoint the exact source of performance erosion.

E. Reporting: The reports are the final product. They must be easily accessible (dashboards, PDF/Excel) and include essential metadata (date, data used, code version) to be audit-ready.

4. System Architecture and Workflows:

This section gives a conceptual view of the system's design and how its components interact.

4.1 Components

The system is a classic data pipeline:

Data Ingestion: Gathers data from various sources.

Storage: A central database for all historical and current data.

Compute: The "brain" where all the heavy lifting (calculations, simulations, optimizations) occurs.

Scheduler: The "engine" that automates the daily workflow.

API & UI: The "interface" that allows users to interact with the system and view results.

4.2 Workflow Examples

These step-by-step examples bring the abstract components to life. For instance, the "Daily Risk Job" workflow shows a sequence: data is loaded, calculations are run, alerts are generated, and a report is published. This makes the system's operation transparent and easy to understand.

5. Testing & Acceptance Criteria: Ensuring Quality

This section outlines the plan for quality assurance. It defines different types of tests—unit tests for mathematical accuracy, integration tests for end-to-end functionality, and User Acceptance Tests (UAT) with real data and users. The acceptance criteria provide a clear, measurable definition of success: "accurate reports delivered on time for 99% of days."

6. Deployment & Maintenance: The Lifecycle

This section covers the practical aspects of putting the system into production and keeping it running smoothly. It includes plans for deployment on a secure platform (servers or cloud), monitoring for errors, data encryption for security, and scheduling upgrades during off-hours to avoid disrupting trading.

7. Future Enhancements: The Roadmap

This section prevents the initial project from becoming over-engineered by listing potential future features. This manages user expectations and provides a clear roadmap for subsequent development phases, such as real-time intraday risk monitoring or support for new asset classes like options and cryptocurrencies.

8. Appendices: The Details

The appendices contain supplementary information that is too detailed for the main document. They serve as a quick reference for technical users.

Appendix A & B: Provide high-level explanations of the VaR and ES formulas, essential for understanding the risk calculations.

Appendix C: Lists specific stress test scenarios, like a "sector crash (–30% sector)," which provides concrete examples of the system's capabilities.

Appendix D: A visual or textual representation of a sample report layout, showing users exactly what the final output will look like.

Appendix E: A detailed glossary with additional terms, such as Alpha (return from skill) and Beta (return from market exposure), providing a more comprehensive understanding of the financial concepts.

This detailed SRS provides a complete and unambiguous guide for the development of the RPAS, ensuring that all parties involved are aligned and that the final product will meet the needs of the business.